Instituto Superior de Engenharia
Politécnico de Coimbra

Advanced Programming

2025/2026

Practical Work

Stage 3 – Java FX graphical interface

1. Objectives

At this stage, the development focus will be on implementing the User Interface (UI) of the "Deep Sea Mining" game in JavaFX. The UI update should conform to the asynchronous notification standard studied in class. The user interfaces are expected to be appealing, intuitive, and functional. The use of fxml is not permitted in the context of this game.

2. Requirements

The goal is to have a user interface (UI) that allows managing all stages of the game, from the option to start a new game to step-by-step progress within the game. The UI should also offer game saving and restoration functionalities, as well as the display and saving of logs generated during gameplay.

User interaction with the game can be via mouse or keyboard shortcuts for selecting operations and controlling game elements.

To display the grids for the various stages of the game (surface, trench, bottom, and minigame), you should only use one element of the type Canvas for each moment, which will occupy the central part of the window and in which all the elements corresponding to that moment of the game will be drawn. To do this, you must define new *nodes* derived from Canvas to implement the functionalities for each moment, and they should be defined following the model of RootPane from MVC@PA. These components should be placed on top of each other on the screen using a StackPane. Only the component corresponding to the active moment should be visible on the screen. The elements of each grid (ship, drone, rock, current, animal, monster, ore, and artifact) should be represented on the screen through images. All images should be stored in the folder images existing in `src/main/resources`.

There should be sidebars to display general game information (ship fuel, drones, fuel levels, hull integrity level, ores, recovered artifacts, etc.). You can use whichever JavaFX elements you deem most appropriate to create these.

There should be a menu in the application (MenuBar) that allows you to present and trigger the following options:

- Game
  - *New* – Start a new game;
  - *Open …* – Restore a previously saved game using a window `FileChooser`;

    - *Open recent* – A submenu should be displayed showing the last 5 recorded games;
    - *Save as …* – save the game, specifying the name and directory through a window `FileChooser`;
    - *Exit*- quit the game;
- *Log*
    - *Show/Hide*– show or hide another window (`Stage`) where the logs generated by the data model are shown;
    - *Save logs* – record the logs, indicating the name and directory through a window `FileChooser`;
    - *Clear logs* – Clear all current logs in the data model.

All data should be obtained from the Facade of the data model (`DeepSeaManager`), including the logs. The Facade must implement the necessary mechanisms so that the UI can receive notifications regarding changes to the model data and logs. This functionality should be implemented using *Property Changes*.

The main classes should be documented with *Javadoc* (you should not generate the *Javadoc* into the project directory, to avoid unnecessary overload on the remote repository).

3. Notes on carrying out the stage

Submission deadline: 2026.05.17 23:59

Management and submission: GitHub+Classroom, using the repository created for each group.

Branch name: StageUI

By the deadline, the code must be integrated into the main branch using **Squash and Merge**. Each group should have only one Pull Request for the branch of the stage, and the Squash & Merge **must** be done on the GitHub website.

You must not alter or delete any branch that has been created and submitted for evaluation, except in the presence of a lecturer from the course unit.

Students must define rules within each group for making commits that reflect a balanced distribution of tasks among group members.(In other words, we don't want to see one group member making 20 commits to change "prints" or "readme" files while another makes only one commit corresponding to the creation of 20 classes). Students working individually should also make regular commits, not just one per stage.